

MOBILE LLM INFERENCE AND INTELLIGENT SELCECTION - ENERGY AWARENESS

PHASE I REPORT

Submitted by
Monic Auditya A (230701194)

Atchaya S (230701045)

Ramya Sreevarshini B (230701262)

Saranya M (230701510)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



**RAJALAKSHMI ENGINEERING COLLEGE,
CHENNAI ANNA UNIVERSITY: CHENNAI 600 025**

December 2026

BONAFIDE CERTIFICATE

Certified that this Report titled “**MOBILE LLM INFERENCE AND INTELLIGENT SELECTION - ENERGY AWARENESS**” is the bonafide work of **Monic Auditya A (230701194), Atchaya S (230701045) Ramya Sreevarshini B (230701262), Saranya M (230701510)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

“This project addresses the following Sustainable Development Goal(s):
SDG 9 : Industry, Innovation and Infrastructure.”

SIGNATURE

Dr. J. Manoranjini, Professor
and Head,
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Karthikeyan N,
Associate Professor
Department of Computer
Science and Engineering,
Rajalakshmi
Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on

Internal Examiner

External Examiner

ABSTRACT

Groups of smartphone users can benefit from a shared artificial intelligence (AI) chat service that operates without cloud connectivity. However, phones differ in processing capability, available memory, battery charge, model availability, and thermal condition. Assigning a conversation permanently to one device or rotating requests indiscriminately across all devices does not account for these differences or for changes during sustained inference.

This report presents the design of a device-aware routing system for offline group AI chat over a local Wi-Fi network. Each participating phone publishes lightweight telemetry, which is normalized into a Live Compute Index (LCI). The router combines this index with model compatibility and admission checks to choose a suitable inference host. An event-based monitor responds to thermal pressure, memory shortage, battery limits, queue growth, and disconnection. A relative stickiness margin of 8% is proposed to suppress unnecessary host changes, while a prepare-ready-route protocol coordinates handoff between compatible devices.

The architecture separates active serving from idle-time calibration and includes mesh communication, LCI computation, event-based routing, host inference, and dash-board modules. It emphasizes transparent decisions, local processing, explicit failure handling, and user control over participation. The implementation chapter specifies a reproducible evaluation plan covering routing latency, sustained throughput, energy per token, prompt-length scalability, and fault tolerance.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Karthikeyan N**, Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Mr. Karthik , B.E., M.E.**, Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

Monic Auditya A (230701194)

Atchaya S (230701045)

Ramya Sreevarshini B (230701262),

Saranya M (230701510)

LIST OF TABLES

3.1	Software Requirements of the Proposed System	14
3.2	Hardware Requirements of the Proposed System	15
5.1	Testbed Hardware Configuration	28
5.2	Routing Performance and Reliability Comparison	30
5.3	Scalability Across Prompt Lengths	31
5.4	Endurance and Fault Tolerance Under 60-Minute Workload	33

LIST OF FIGURES

1.1	Overview of the Proposed System	3
3.1	System Architecture, Phone Roles over the Local Mesh	12
3.2	Activity Diagram for the Router Environment	18
3.3	Activity Diagram for the Host Environment	19
3.4	Data Flow Diagram of the System	20
4.1	LCI Telemetry and Candidate Scoring Pipeline	23
4.2	Event-Based Routing State Machine	25
5.1	Thermal Degradation and Energy per Token	28
5.2	60-Minute Throughput and Thermal Dynamics	30
5.3	Latency CDF and Tail Percentiles	32
5.4	Multi-Dimensional System Performance Trade-off Radar	33

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CPU	Central Processing Unit
DVFS	Dynamic Voltage and Frequency Scaling
GPU	Graphics Processing Unit
HAL	Hardware Abstraction Layer
IPC	Inter-Process Communication
JNI	Java Native Interface
LCI	Live Compute Index
LLM	Large Language Model
MEC	Mobile Edge Computing
MTBF	Mean Time Between Failures
NPU	Neural Processing Unit
OOM	Out of Memory
RAM	Random Access Memory
RTT	Round Trip Time
SoC	System on Chip
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TTFT	Time to First Token
UML	Unified Modeling Language
Wi-Fi	Wireless networking technology

TABLE OF CONTENTS

ABSTRACT	i
LIST OF TABLES	ii
LIST OF FIGURES	iii
LIST OF ABBREVIATIONS	iv
1 Introduction	1
1.1 General.....	1
1.2 Objectives.....	2
1.3 Existing System.....	2
1.4 Proposed System.....	3
2 Literature Review	5
2.1 Literature Review.....	5
2.2 Comparison and Discussion.....	9
2.3 Conclusion.....	10
3 System Design	11
3.1 System Architecture.....	11
3.2 System Requirements.....	13
3.2.1 Software Requirements.....	13
3.2.2 Hardware Requirements.....	14
3.2.3 Security Requirements.....	15
3.2.4 Deployment and Scaling.....	16
3.3 System Design.....	17
3.3.1 Activity Diagram.....	17
3.3.2 Data Flow Diagram.....	19
4 Project Description	21
4.1 Technologies.....	21
4.2 Modules.....	21
4.2.1 Mesh Communication Module.....	22
4.2.2 LCI Computation Module.....	22
4.2.3 Event-Based Routing Module.....	24
4.2.4 Host Inference Module.....	25
4.2.5 Dashboard.....	26
5 Implementation and Result	27
5.1 Telemetry Preprocessing Module.....	27
5.1.1 Telemetry Collection and Preprocessing.....	27
5.1.2 Implementation of LCI Computation.....	29
5.1.3 Implementation of Event-Based Routing.....	30
5.2 Dashboard.....	32
6 Conclusion and Future Work	34
6.1 Conclusion.....	34
6.2 Future Work.....	34
REFERENCES	36

ABSTRACT

Groups of smartphone users can benefit from a shared artificial intelligence (AI) chat service that operates without cloud connectivity. However, phones differ in processing capability, available memory, battery charge, model availability, and thermal condition. Assigning a conversation permanently to one device or rotating requests indiscriminately across all devices does not account for these differences or for changes during sustained inference.

This report presents the design of a device-aware routing system for offline group AI chat over a local Wi-Fi network. Each participating phone publishes lightweight telemetry, which is normalized into a Live Compute Index (LCI). The router combines this index with model compatibility and admission checks to choose a suitable inference host. An event-based monitor responds to thermal pressure, memory shortage, battery limits, queue growth, and disconnection. A relative stickiness margin of 8% is proposed to suppress unnecessary host changes, while a prepare–ready–route protocol coordinates handoff between compatible devices.

The architecture separates active serving from idle-time calibration and includes mesh communication, LCI computation, event-based routing, host inference, and dashboard modules. It emphasizes transparent decisions, local processing, explicit failure handling, and user control over participation. The implementation chapter specifies a reproducible evaluation plan covering routing latency, sustained throughput, energy per token, prompt-length scalability, and fault tolerance.

The supplied five-phone evaluation reports 22.8 tokens/s sustained decode throughput, 8.45 s p95 request latency, and 99.4% request success for adaptive routing. Its endurance summary reports a 38.4 °C peak battery temperature and 142 ms handoff latency. The report presents these source-reported results with the supplied diagrams and plots; conclusions are bounded by the workload and measurement definitions.

LIST OF TABLES

3.1	Software Requirements of the Proposed System	14
3.2	Hardware Requirements of the Proposed System	15
5.1	Device Configuration in the Supplied Five-Phone Evaluation	30
5.2	Reported Inference Throughput and Request Latency	32
5.3	Reported Tail Latency, Thermal Metric, and Request Success	32
5.4	Reported Time to First Token across Prompt Lengths	34
5.5	Reported Prefill Throughput across Prompt Lengths	34
5.6	Reported Endurance, Battery Temperature, and Failure Outcomes . . .	36
5.7	Reported Handoff Outcomes and Availability Statistics	36

LIST OF FIGURES

1.1	Overview of the Proposed System	3
3.1	System Architecture, Phone Roles over the Local Mesh	12
3.2	Activity Diagram for the Router Environment	18
3.3	Activity Diagram for the Host Environment	19
3.4	Data Flow Diagram of the System	20
4.1	LCI Telemetry and Candidate Scoring Pipeline	24
4.2	Event-Based Routing State Machine	26
5.1	Thermal Degradation and Source-Reported Energy per Generated Token	30
5.2	Sustained Throughput and Battery-Temperature Dynamics during the Continuous Session	33
5.3	Request-Latency Cumulative Distribution and Tail Percentiles	35
5.4	Source-Provided Multi-Dimensional Performance Trade-off Analysis .	37

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CPU	Central Processing Unit
DVFS	Dynamic Voltage and Frequency Scaling
GPU	Graphics Processing Unit
HAL	Hardware Abstraction Layer
IPC	Inter-Process Communication
JNI	Java Native Interface
LCI	Live Compute Index
LLM	Large Language Model
MEC	Mobile Edge Computing
MTBF	Mean Time Between Failures
NPU	Neural Processing Unit
OOM	Out of Memory
RAM	Random Access Memory
RTT	Round Trip Time
SoC	System on Chip
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TTFT	Time to First Token
UML	Unified Modeling Language
Wi-Fi	Wireless networking technology

TABLE OF CONTENTS

ABSTRACT	i
LIST OF TABLES	ii
LIST OF FIGURES	iii
LIST OF ABBREVIATIONS	iv
1 Introduction	1
1.1 General	1
1.2 Objectives	2
1.3 Existing System	2
1.4 Proposed System	3
2 Literature Review	5
2.1 Literature Review	5
2.2 Comparison and Discussion	9
2.3 Conclusion	10
3 System Design	11
3.1 System Architecture	11
3.2 System Requirements	13
3.2.1 Software Requirements	13
3.2.2 Hardware Requirements	14
3.2.3 Security Requirements	15
3.2.4 Deployment and Scaling	16
3.3 System Design	17
3.3.1 Activity Diagram	17
3.3.2 Data Flow Diagram	19
4 Project Description	21
4.1 Technologies	21
4.2 Modules	21
4.2.1 Mesh Communication Module	22
4.2.2 LCI Computation Module	22
4.2.3 Event-Based Routing Module	25
4.2.4 Host Inference Module	26
4.2.5 Dashboard	27
5 Implementation and Results	29
5.1 Telemetry Preprocessing Module	29
5.1.1 Telemetry Collection and Preprocessing	29
5.1.2 Implementation of LCI Computation	30
5.1.3 Implementation of Event-Based Routing	33
5.2 Dashboard	35
6 Conclusion and Future Work	38
6.1 Conclusion	38
6.2 Future Work	38
REFERENCES	40

CHAPTER 1

INTRODUCTION

1.1 GENERAL

The availability of compact language models on consumer smartphones provides an opportunity for groups of people to access AI assistance without depending on cloud servers or a stable internet connection. A shared local session can support collaborative learning, discussion, and assistance in low-connectivity environments. Keeping inference within a trusted local network also reduces dependence on external services and gives participants greater control over where their conversations are processed.

The ability to execute a model locally does not imply that every phone can sustain the same workload. Devices differ in chip generation, memory capacity, model support, cooling characteristics, battery charge, and background activity. A phone that performs well at the beginning of a session may become less suitable after repeated generation requests. Conversely, a device that was initially busy may later become available.

Static assignment and indiscriminate rotation can therefore produce inconsistent response times. A low-memory device may fail an admission check or encounter an out-of-memory condition. A high-performance device may slow under sustained thermal pressure. Repeatedly changing hosts can also discard useful runtime state and require the new host to reconstruct conversation context. Routing must consider both the benefit of selecting a better device and the cost of moving a session.

The proposed system treats host selection as a continuously monitored decision rather than a one-time benchmark result. Lightweight telemetry describes the current state of each phone, while an event-based policy determines when the present assignment should be reconsidered. The objective is a responsive and explainable local group-chat service with explicit limits on resource consumption and clear behavior when no suitable host is available.

In this report, the term *mesh* denotes the logical network of cooperating peer phones. It does not assume that ordinary Android phones implement a multi-hop Wi-Fi mesh at the link layer. The initial deployment may use a shared local access point or a supported peer-to-peer arrangement without an internet uplink.

1.2 OBJECTIVES

The principal objective is to design real-time, device-aware routing of AI chat queries across a group of locally connected phones. Host selection must combine current compute availability, model compatibility, and operational stability rather than depend on a single static ranking.

The specific objectives are to:

1. Discover trusted participating phones and maintain a current registry of capabilities, available models, and serving roles.
2. Develop an LCI that combines decode speed, memory headroom, thermal headroom, queue availability, battery state, and network RTT in a transparent score.
3. Apply admission guards before ranking candidates so that a high score cannot override an incompatible model, unsafe condition, or inadequate memory.
4. Reconsider assignments when relevant device or network events occur, with a stickiness margin to reduce unnecessary handoffs.
5. Coordinate conversation handoff through a prepare-ready-route sequence with explicit timeout, rollback, and recovery behavior.
6. Provide a dashboard that explains scores, exclusions, host changes, and resource conditions to participants.
7. Evaluate sustained performance, energy use, reliability, and scalability using reproducible workloads and appropriate baselines.

The supplied design proposes a telemetry heartbeat of one second, a lightweight score/probe budget below 80 ms, and a warm-host routing-control handoff target below 200 ms. These values are preliminary engineering targets, not demonstrated performance guarantees. Model loading, context reconstruction, and reconnection are measured separately and may take substantially longer.

1.3 EXISTING SYSTEM

Existing mobile and edge inference systems address scheduling at several levels. Some select a processor within one phone, others offload tasks to an edge server, and distributed frameworks can partition one model across several devices. These approaches solve related resource-management problems, but their scheduling units and deployment assumptions differ from whole-session routing among peer phones.

Static host assignment. A designated high-capability phone serves all requests.

This is simple to implement and preserves a warm model cache, but its suitability is not revisited as temperature, battery charge, or background load changes.

Round-robin assignment. Requests rotate through the participating devices. This shares work mechanically, but does not account for whether each device can fit the model or deliver an acceptable service time. Conversation-state synchronization may also become expensive.

Queue- or latency-based dispatch. The scheduler chooses a device with a short queue or a low network RTT. These signals capture part of the service cost but do not, by themselves, describe memory suitability or sustained inference capability.

Resource-aware scheduling. Thermal-aware and battery-aware approaches introduce richer device-state information. The reviewed works demonstrate relevant mechanisms, although several operate within one device or depend on a separate edge-computing layer. Their scope is examined in Chapter 2.

Two extremes are undesirable for the proposed setting. Never revisiting the initial assignment can leave a deteriorating host in service, while recomputing and changing the assignment for every query can incur excessive switching cost. The proposed design therefore combines periodic observations with event-driven decisions and explicit hysteresis.

1.4 PROPOSED SYSTEM

The proposed system integrates mesh communication, an LCI engine, candidate admission and scoring, an event monitor, a coordinated handoff protocol, and a participant dashboard. Every phone reports its local state; the router selects one feasible host for an entire conversation rather than partitioning each generation across multiple phones.

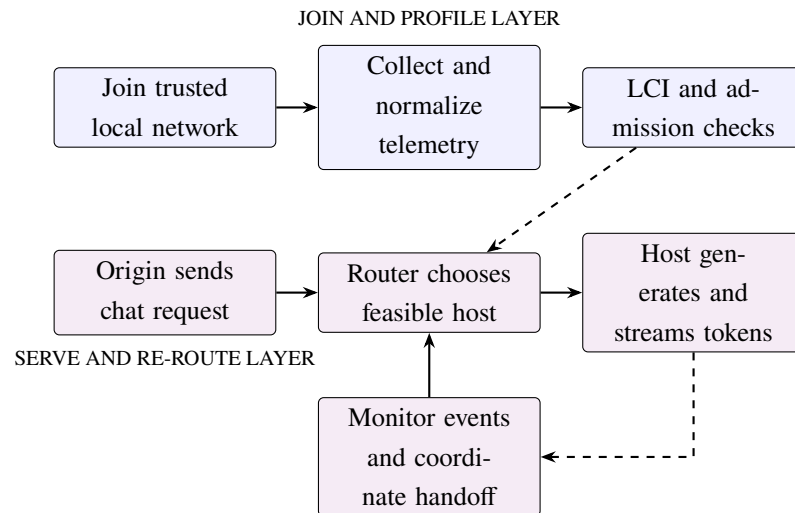


Figure 1.1: Overview of the Proposed System

Figure 1.1 distinguishes the join-and-profile layer, shown in blue, from the serving-and-re-routing layer, shown in violet. Telemetry is updated through a proposed one-second heartbeat, while active requests use the latest valid device snapshot. Joining, leaving, and changing roles are managed without assuming that all devices remain continuously available.

The LCI represents current resource suitability. A separate feasibility guard checks memory, model identity, freshness, consent, and safety conditions. The event monitor requests a new selection when the current host becomes unsuitable or a materially better candidate appears. A relative 8% improvement margin is used for elective changes; emergency recovery bypasses that margin when the existing host is no longer feasible.

All inference and routing are intended to remain within the local network. This does not, by itself, guarantee confidentiality: participants must authenticate peers, protect transport, and understand that the selected host can access conversation content. The security requirements in Chapter 3 address these boundaries.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the supplied research themes and relates them to the proposed scheduling problem. The comparison distinguishes processor scheduling, task offloading, distributed model execution, and whole-session host routing. Publication results are evidence for their own experimental settings, not measured outcomes of this project.

2.1 LITERATURE REVIEW

THERMAL-AWARE SCHEDULING ON MOBILE DEVICES

Tan and Cao [1] investigate thermal-aware scheduling of deep neural networks on mobile devices with GPUs and NPUs. They propose a heuristic scheduler and a reinforcement-learning-based extension to account for future consequences of scheduling decisions. The work motivates including thermal state rather than relying solely on instantaneous speed. Its scheduling scope is the processing resources within one mobile device; choosing among independently owned phones and transferring an ongoing conversation are outside that scope.

For the proposed system, the relevant principle is that immediate throughput is not a sufficient measure of sustainable service. Thermal state should influence both candidate ranking and hard safety exclusions. The project adopts this principle without assuming that the study’s processor-level policy directly applies to inter-device routing.

TEMPERATURE-AWARE HETEROGENEOUS COMPUTING

Gao’s TAS framework [2] considers temperature-aware scheduling for heterogeneous computing resources. The supplied literature summary describes a temperature perception model, speed control, task scheduling, and assignment optimization for CPU, GPU, and NPU resources. This provides a related example of thermal constraints influencing scheduling decisions.

The distinction is the scheduling unit. TAS concerns heterogeneous processors and their task allocation, whereas this project selects a physical phone that already holds a compatible model. The proposed router consequently includes peer reachability, battery participation limits, and conversation-state readiness in addition to local resource information.

THERMAL-AWARE SINGLE-LAYER PARALLELIZATION

Zhang et al. [3] combine thermal-aware dynamic frequency adjustment with heterogeneous single-layer workload distribution. Their approach adjusts processor operating speed and distributes neural-network work according to processor and layer characteristics. The study focuses on controlling on-device execution, including environments in which ambient temperature matters.

This work supports separating sustained capability from nominal hardware specifications. However, the proposed project does not split an individual neural-network layer between phones. Its main decision is whether a complete conversation should continue on the current host or move to another feasible device.

MULTI-DNN PROCESSOR CO-EXECUTION

Gao et al. [4] propose Advanced Multi-DNN Model Scheduling for heterogeneous mobile processors. Their method combines offline subgraph partitioning with a processor-state-aware runtime algorithm to distribute work under changing execution conditions. It addresses the scheduling complexity and memory overhead associated with heterogeneous co-execution.

The relevance to this project is the use of live state instead of an immutable capacity label. The difference is that the proposed mesh router works at the device and conversation levels. It need not reproduce the paper’s subgraph partitioning, but it must ensure that each selected host can execute the requested model independently.

SUSTAINED LLM INFERENCE AT THE EDGE

Tummalapalli et al. [5] characterize LLM inference across mobile, NPU, and GPU platforms under sustained load. Their study measures throughput, latency, power, and thermal behavior for a fixed model and workload. The authors emphasize that the results describe hardware and software together and should not be interpreted as general rankings of hardware capability alone.

This directly informs the evaluation design: a cold-start benchmark is insufficient to characterize a long group-chat session. Measurements must include repeated requests, warm operation, thermal evolution, and energy per generated token. The proposed system’s improvement must be established on its own testbed rather than inferred from another platform’s behavior.

NETWORK-EDGE LLM INFERENCE SURVEY

Chen et al. [6] survey network-edge inference for large language models, including architectures, model optimization, deployment, resource management, and scheduling. The survey places the proposed project within a broader research area shaped by constrained compute, memory, and communication resources.

The project addresses a narrower practical setting: a trusted group of phones, a local network without required internet access, compatible independently executable models, and transparent whole-session routing. This is a design focus, not proof that no related system exists. A comprehensive novelty claim would require a broader systematic review than the sources considered here.

LINGUALINKED: DISTRIBUTED MOBILE LLM INFERENCE

Zhao et al. [7] introduce LinguaLinked, a decentralized mobile LLM inference system. It partitions a model, assigns segments according to device capabilities, optimizes transmission between segments, and includes runtime load balancing. This enables collaborative execution when a model cannot be handled conveniently by one device.

The proposed system instead assumes that at least one feasible phone can run the requested model independently. It routes a complete conversation to one such phone and treats handoff as a change of host. Model parallelism and host selection are complementary approaches, but their network costs and failure modes differ. A segment failure can disrupt a distributed execution chain, while a host-routing design must reconstruct or transfer the conversation to another independent executor.

DEEP REINFORCEMENT LEARNING FOR OFFLOADING

Lim et al. [8] propose DRL-OS, an offloading scheduler based on a double dueling deep Q-network. It uses task size, deadline, queue state, and residual battery charge to choose between local execution, remote offloading, and dropping a task. The study demonstrates a state-dependent scheduling formulation rather than a fixed local-or-remote policy.

For this project, the relevant lesson is that battery state and queue pressure influence service suitability. However, a trained policy introduces a separate model-development and validation process. The proposed LCI is deliberately transparent and arithmetic-based. Its overhead still requires measurement; it should not be assumed to meet the probe budget simply because its formula is lightweight.

BATTERY-CARE RESOURCE ALLOCATION AFTER DISASTERS

Tang et al. [9] study battery-care resource allocation and task offloading in a post-disaster mobile edge environment. They consider unstable connectivity, device-to-device communication, delay, energy consumption, and battery degradation using a multi-agent optimization formulation. The supplied draft attributed this work to Zhan; the reference here uses Tang, the first listed author.

The connection to the proposed system is the importance of connectivity changes and battery participation limits. Nevertheless, the post-disaster MEC setting and its optimization objectives are different from a lightweight group-chat host router. The project adopts explicit disconnect handling and configurable battery eligibility without claiming that charge alone measures long-term battery health.

FUZZY PRIORITY-AWARE TASK SCHEDULING

Lin [10] presents fuzzy-based priority-aware task scheduling for mobile edge computing. The framework combines Greedy Auto-Scaling Offloading with fuzzy scheduling and resource allocation at the edge. It uses task execution time and energy considerations to assign priorities within a multi-tier environment.

This illustrates another way to combine multiple scheduling criteria. The proposed LCI uses a weighted, normalized score with separately stated exclusion rules rather than a fuzzy rule base. A weighted score is easier to display, but its weights and normalization still need calibration. Transparency does not eliminate the need for workload-specific evaluation.

PEER-TO-PEER INFERENCE RUNTIMES

The Mesh LLM project [11] documents distributed inference across networked machines and provides operational tooling for shared model serving. It is useful as a practical example of networked inference, but its deployment should not be treated as an Android smartphone experiment.

The proposed project's choice of whole-session routing avoids requiring every generated token to cross a sequence of model partitions. Whether this yields a performance advantage depends on the model, host hardware, network, and handoff frequency. The exact throughput figures in the supplied draft are not reused as smartphone-mesh results because the corresponding benchmark configuration was not supplied with the project.

2.2 COMPARISON AND DISCUSSION

The reviewed works agree on the importance of device and workload state, but they operate at different layers. Processor-level methods improve execution inside one phone. Offloading policies choose between a local device and an external compute layer. Distributed model frameworks make several devices cooperate on one model execution. These are not interchangeable with selecting one peer phone for a complete conversation.

Scheduling granularity. Thermal and heterogeneous-processor studies act on processors, tasks, or neural-network subgraphs. LinguaLinked partitions models across devices. The proposed router acts on a conversation-host assignment, preserving one active executor for each session.

Decision inputs. The reviewed scheduling work considers thermal state, processor status, queues, deadlines, and battery constraints. The proposed design combines six normalized signals, but also uses hard eligibility conditions. Model compatibility and memory fit are not merely soft preferences.

Decision overhead. A weighted composite score has predictable arithmetic cost and can be inspected by users. However, probe acquisition, peer communication, and context transfer may dominate the total overhead. The evaluation must measure these stages separately instead of reporting score computation alone as end-to-end routing latency.

Stability and adaptation. Continuous monitoring is useful only if it produces appropriate action. Minor score fluctuations should not cause repeated switching. The proposed design therefore includes a relative improvement margin, a configurable persistence window, and a minimum dwell time for elective handoffs. Safety failures and disconnected hosts use a separate recovery path.

Privacy and trust. Removing cloud dependence changes the trust boundary; it does not remove it. The selected peer receives conversation content. Consent, authentication, access control, and protected local transport are consequently part of the proposed system rather than optional additions.

The contribution pursued here is the integration of live device scoring, admission guards, stable event-driven host selection, and conversation-level handoff in a local smartphone group. The reviewed material motivates this combination but does not establish exhaustive uniqueness or measured superiority.

2.3 CONCLUSION

The literature provides substantial support for resource-aware scheduling on constrained devices. Thermal studies motivate live headroom measurements; offloading work highlights battery and queue state; distributed inference demonstrates collaboration between devices; and sustained-load benchmarking emphasizes the need to evaluate long sessions rather than startup behavior alone.

The proposed design focuses on one independently executable host per conversation. It must remain robust when a phone becomes unavailable, overheats, encounters memory pressure, or withdraws consent. A transparent score is combined with hard admission conditions and a coordinated handoff protocol so that adaptation does not become uncontrolled switching.

The design targets in the supplied material, including the one-second heartbeat, sub-80 ms lightweight decision budget, and sub-200 ms warm routing-control handoff, remain hypotheses for validation. Their feasibility depends on actual device behavior, model readiness, context length, and network conditions.

CHAPTER 3

SYSTEM DESIGN

This chapter describes the proposed architecture, software and hardware requirements, security controls, deployment strategy, and system-flow diagrams. The design replaces the unrelated Reddit moderation, server-training, and cloud data-pipeline material present in the supplied template with requirements appropriate to local smartphone inference.

3.1 SYSTEM ARCHITECTURE

The system is organized into a serving environment and a calibration environment. Serving handles discovery, requests, routing, token streaming, event monitoring, and handoff. Calibration updates device-specific estimates during suitable idle periods so that benchmarking does not compete with an active conversation.

Every participant can take one or more logical roles:

Origin: Captures the user’s request, keeps the authoritative conversation transcript, displays streamed responses, and records completed turns.

Router: Maintains current peer telemetry and host assignments, applies eligibility checks, ranks candidates, and coordinates handoff.

Host: Executes the selected local model, manages context and runtime memory, and streams generated output.

Member: Participates in discovery and telemetry and may become a host when eligible and permitted by its owner.

These are logical roles rather than separate hardware classes. One phone can be both an origin and a host. The initial design uses one active routing authority per group and an assignment epoch to identify its decisions. Recovery from router loss requires a replacement authority and must not create two simultaneously valid host assignments.

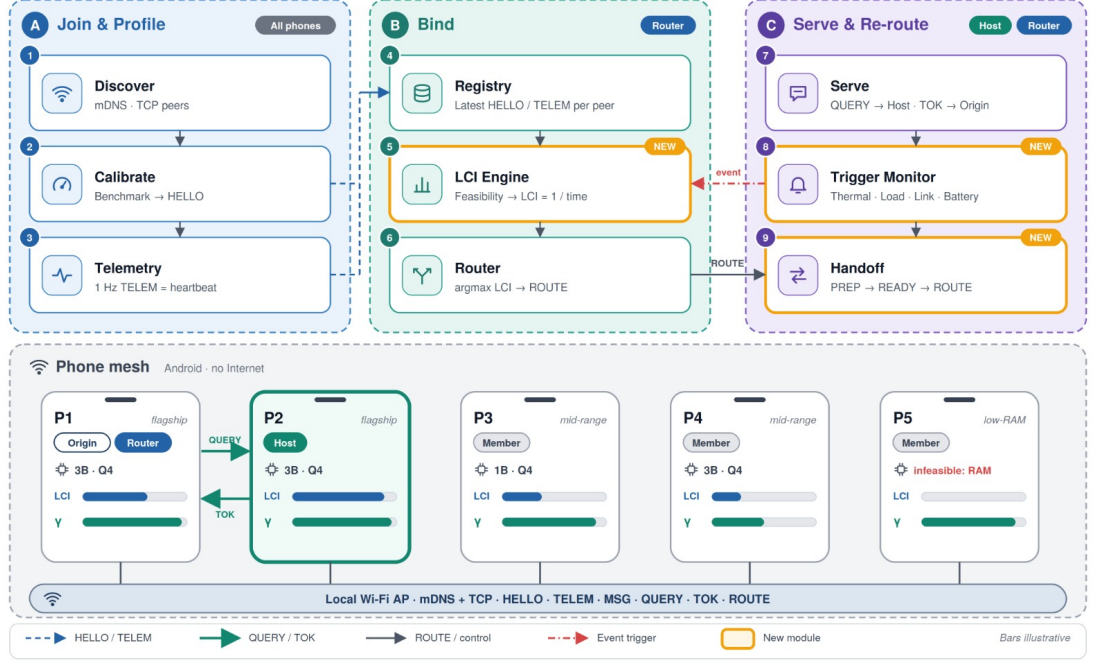


Figure 3.1: System Architecture, Phone Roles over the Local Mesh

Figure 3.1 shows how the origin, router, host, and member roles exchange data over the local network. Requests and output belong to the serving path. Telemetry and event logs also feed an idle calibration path, but calibration never needs to upload conversation content.

Serving environment. A telemetry coroutine collects supported device-state observations, timestamps each snapshot, and publishes it to the router. The router evaluates the requested model and expected context memory against the latest valid state. It either retains an eligible host, selects a new host, or returns an explicit unavailable outcome. A bounded queue prevents unbounded accumulation of requests.

Event monitoring and handoff. The monitor distinguishes elective optimization from forced recovery. Elective switching requires a materially better feasible candidate. Forced recovery responds to disconnection, consent withdrawal, critical thermal state, or failed memory admission. The prepare stage checks model and context readiness; ready confirms admission; route commits a new assignment epoch. Late output from an old assignment is not accepted as a second response.

Calibration environment. A short local benchmark estimates prefill and decode capability for the supported runtime and model. Actual service-time history can be compared with these estimates to identify drift. A simple regression or correction factor may be fitted locally, provided enough valid samples exist. Calibration constants are replaced atomically during an idle window and are versioned so that the router can explain which profile informed its decision.

The telemetry buffer stores bounded observations and routing events, not an unlimited record of private chats. Conversation transcripts remain under origin control unless the user explicitly chooses a retention or export policy. When no alternative host is feasible, the system exposes the reason rather than retrying indefinitely.

3.2 SYSTEM REQUIREMENTS

The proposed system requires an Android client, a compatible native inference runtime, a local communication layer, and enough memory on at least one participating phone to host the selected model. It does not require a Reddit dataset, cloud warehouse, training server, or internet connection during a chat session.

3.2.1 SOFTWARE REQUIREMENTS

Kotlin is proposed as the primary language for the Android application. It provides a consistent client implementation for chat interaction, asynchronous telemetry, and local networking. Performance-critical inference is implemented through a C++ runtime such as llama.cpp and accessed through JNI. Exact supported platform versions and runtime revisions must be recorded during implementation rather than left as moving dependencies.

The native interface exposes model loading, tokenization, context creation, generation, cancellation, and release. Calls must not block the UI thread. The application records model identity, quantization, tokenizer compatibility, and context limits so that the router does not treat different model files as equivalent simply because their names look similar.

Networking uses a framed local protocol over authenticated transport. Discovery can use supported Android local-network mechanisms or explicit peer configuration. The communication module must handle permission denial, unsupported discovery features, network changes, partial reads, timeouts, and reconnect attempts. Offline operation requires models and application assets to be installed before the session begins.

Table 3.1: Software Requirements of the Proposed System

Component	Requirement	Purpose
Android platform	Android release supporting required networking and telemetry APIs	Client execution, permissions, telemetry, and lifecycle handling
Kotlin and coroutines	Pinned compiler and compatible coroutine library	Application logic and asynchronous tasks
C++ and Android NDK	Toolchain compatible with selected runtime revision	Native inference and memory management
Inference runtime	llama.cpp	Quantized model loading and generation
JNI interface	Validated native bindings	Communication between Kotlin and C++
Local networking	Supported discovery; framed authenticated connections	Peer registry, telemetry, requests, and responses
Model assets	Matching model hash, tokenizer, quantization, and context limit	Compatible independent host execution
Local storage	Bounded transcript and telemetry storage	Recovery, diagnostics, and evaluation export
Dashboard UI	Android-native interface	Chat interaction and routing explanations
Development tools	Android build tools and device profiling tools	Build, deployment, debugging, and measurement

Python may be used outside the serving application to analyze exported experimental logs. It is not the proposed Android serving language, and the design does not depend on Anaconda, container orchestration, or an always-on remote server.

3.2.2 HARDWARE REQUIREMENTS

The deployment uses a group of heterogeneous Android smartphones connected through a local Wi-Fi network. Device suitability is evaluated against the chosen model and workload instead of a fixed desktop-style specification. A phone may participate as a client even when it lacks enough memory to act as an inference host.

Host memory must cover model weights, runtime buffers, context state, operating-system demand, and a safety reserve. Installed RAM alone does not establish feasibility because available memory changes with background applications and system activity. Likewise, core count is not a reliable substitute for measured sustained generation speed.

The evaluation should include low-, mid-, and high-capability phones where available, but their actual identities and specifications must be recorded in the testbed table. Devices should use the same compatible model and generation configuration for controlled comparisons. Optional GPU acceleration is only enabled when the selected runtime and device combination support it reliably.

Table 3.2: Hardware Requirements of the Proposed System

Component	Requirement	Purpose
Phone group	At least two compatible phones for handoff; five in the supplied evaluation	Origin, router, host, and alternative host roles
Processor	CPU supported by the native runtime	Local inference and lightweight probes
RAM	Free memory exceeding model, context, buffers, and reserve	Safe admission and OOM avoidance
Storage	Space for model assets and bounded local logs	Offline model availability and diagnostics
Wireless network	Shared local Wi-Fi or supported peer arrangement	Peer communication without required internet access
Battery and power	User-configured participation limit; chargers for controlled tests	Safe participation and endurance experiments
Thermal observations	Available OS thermal status; optional validated sensors	Thermal exclusion and sustained-load evaluation
Development computer	Computer capable of building and deploying the client	Development only; not a production inference server
Power measurement	External meter or documented supported readings	Energy estimates with stated uncertainty

Missing sensor readings must be represented as unavailable, not as zero temperature, unlimited battery, or perfect headroom. A conservative fallback and the source of each observation should be displayed in the dashboard. The system must also respect OS-enforced limits rather than bypass thermal protection.

3.2.3 SECURITY REQUIREMENTS

The principal assets are conversation content, peer identities, model files, assignment authority, and telemetry integrity. The main threats include unauthorized joining, eavesdropping, spoofed telemetry, replayed routing messages, malformed requests, and accidental retention of private data.

Peer admission and consent. A phone joins only after an explicit pairing or approval step. Participants can see the group membership, enable or disable hosting, and withdraw at any time. The design does not assume that every device on the same access point is trusted.

Transport and authentication. Requests, transcripts, and responses use authenticated encrypted connections. Session identifiers, nonces, sequence numbers, and assignment epochs prevent old control messages from being interpreted as current authority. Credentials and pairing material are kept in an appropriate local secure store.

Input and model validation. The application bounds message size, queue length, context length, and request rate. Model hashes and tokenizer identifiers are checked before handoff. Malformed or incompatible peers are rejected with explicit reasons.

Telemetry integrity. The router verifies message identity, freshness, and sequence order. These checks prevent network spoofing but cannot prove that an authenticated phone reports truthful sensor values. The initial system assumes trusted participants; stronger attestation and adversarial scoring are future work.

Retention and access control. Debug exports exclude conversation text by default. Diagnostic logs store only the minimum fields required for analysis. Users control transcript retention and deletion. Offline processing does not conceal content from the chosen inference host, and this limitation must be communicated during consent.

3.2.4 DEPLOYMENT AND SCALING

Deployment begins by installing the Android application and compatible model assets on the intended hosts. Participants create or join a local network, pair devices, and select a routing authority. The router builds its registry from capability advertisements and fresh telemetry before admitting requests.

For a small group of N phones, each member publishes one compact state snapshot per heartbeat. The router keeps the latest valid snapshot and performs a linear scan of eligible candidates. This avoids broadcasting a complete telemetry history or rerunning expensive benchmarks for every query. The one-second cadence is configurable because its power cost and freshness benefit must be evaluated.

Scaling to multiple conversations requires queue accounting, per-host admission, and explicit memory reservation for each context. A host is not admitted merely because it can load the model once. The initial design prioritizes correct operation for a small trusted group; multi-router coordination and large-group scheduling remain extensions.

Router loss is handled through a defined recovery process with monotonically

changing authority epochs. A replacement must reconcile the latest committed transcript and valid assignments before issuing new work. Network partitions may require suspending affected conversations rather than pretending that a single global authority still exists.

3.3 SYSTEM DESIGN

The following diagrams describe intended control and data flow. They are proposed design diagrams, not screenshots of an implemented application. The activity diagrams use entry and exit states, process blocks, decisions, and directed transitions. The data flow diagram describes data movement and stores; it is not a UML activity diagram.

3.3.1 ACTIVITY DIAGRAM

The router environment begins when a request or a routing event is received. It refreshes the relevant state, filters candidates, and evaluates whether the current assignment can be retained. An elective switch must meet the improvement rule; a hard failure instead invokes recovery. If no phone can serve the request, the origin receives a clear unavailable response.

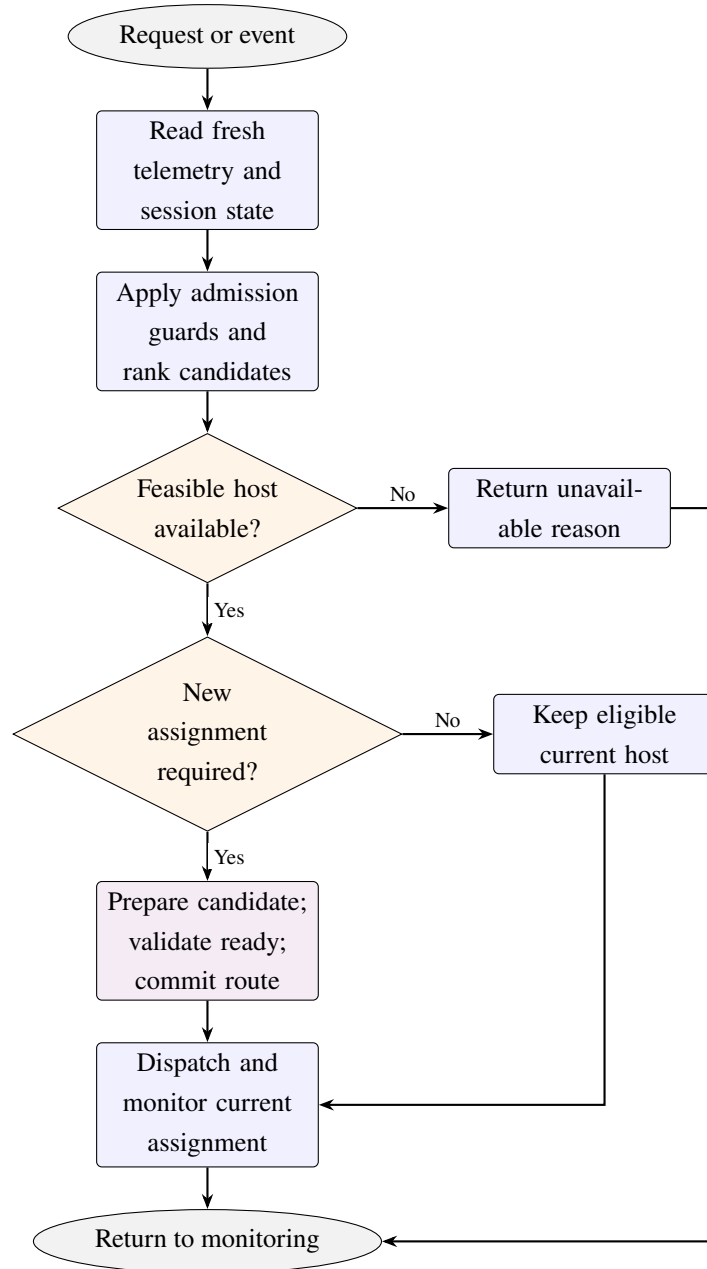


Figure 3.2: Activity Diagram for the Router Environment

Figure 3.2 summarizes the successful preparation path. Preparation failure does not commit a route: the router retains an eligible old host or tries the next feasible candidate within a bounded retry policy. The decision is recorded with its reason and epoch.

The host environment starts with a preparation or serving request. It authenticates the caller, validates model identity, reserves memory, and reconstructs the required context. Only after successful admission does it report readiness. Generation is cancellable and continuously subject to runtime safety checks.

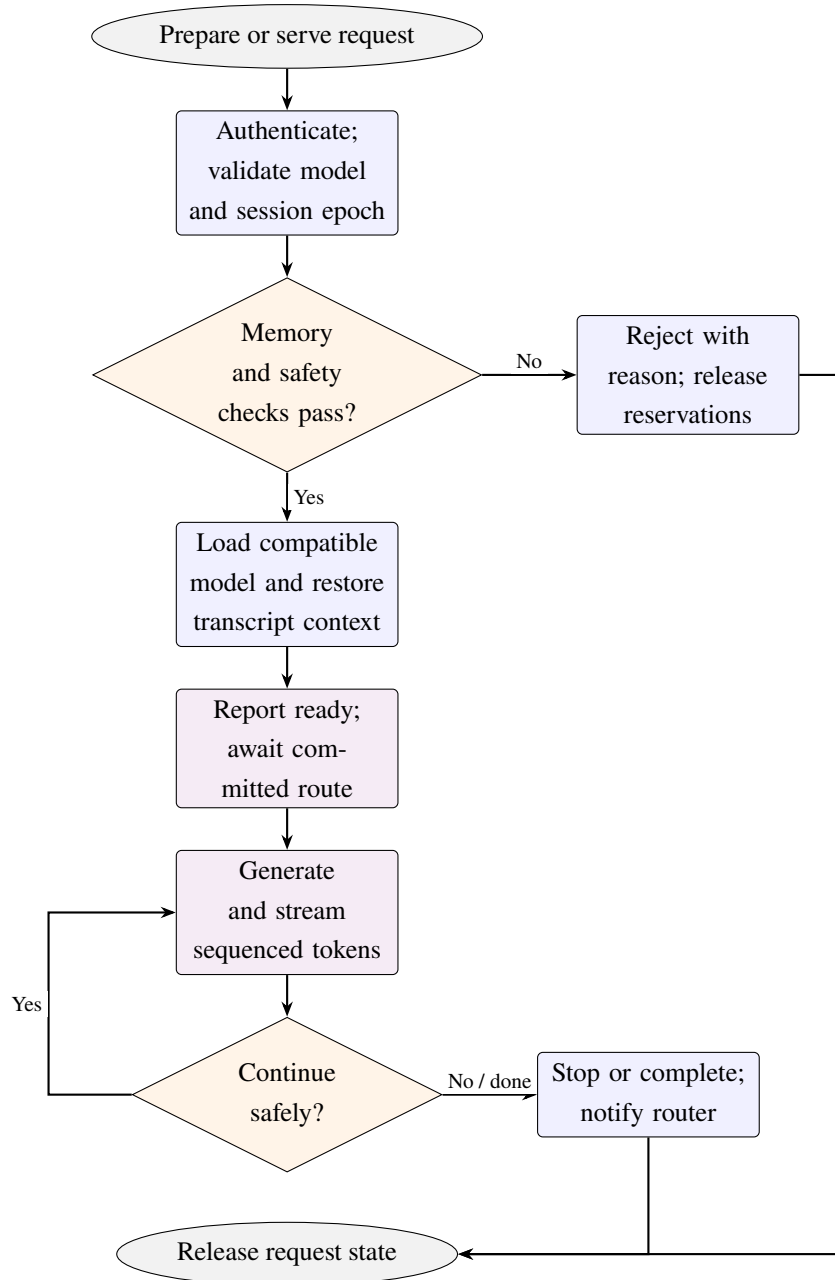


Figure 3.3: Activity Diagram for the Host Environment

Figure 3.3 separates readiness from committed execution. A host must not generate a second authoritative response merely because it has prepared a context. Readiness messages expire, and a failed or cancelled preparation releases reserved memory.

3.3.2 DATA FLOW DIAGRAM

The system has four principal data streams: capability and telemetry messages, requests and transcripts, token responses, and diagnostic events. Conversation data move from the origin to the selected host; device-state data move from participants to the router;

diagnostic summaries are stored locally for calibration and evaluation.

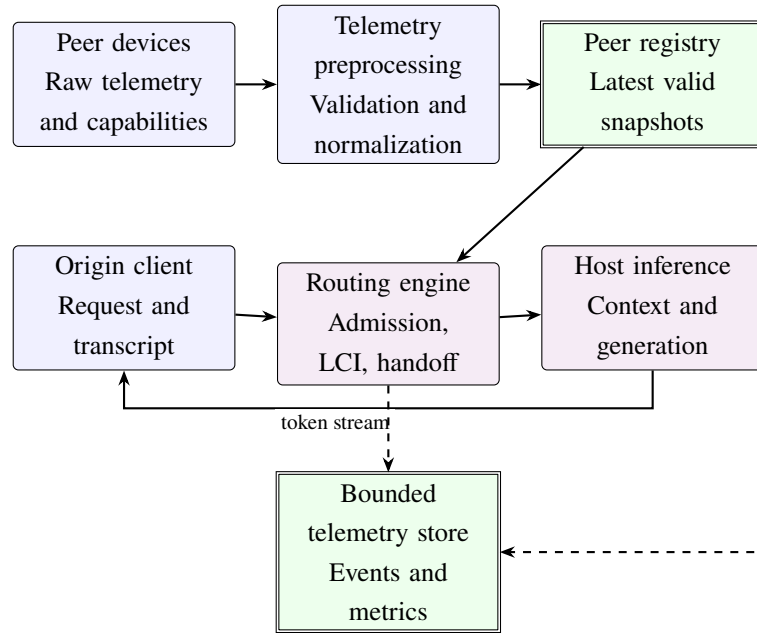


Figure 3.4: Data Flow Diagram of the System

Figure 3.4 is a level-1 logical data flow diagram. Double-bordered stores distinguish persisted state from processing components. The diagram does not imply cloud storage or a training data warehouse. Raw telemetry is transformed into bounded, timestamped observations, while the origin retains the authoritative record of completed conversation turns.

During handoff, the router sends the new host the model identity, assignment information, and authorized conversation context. Context transfer is distinct from ordinary telemetry. The selected host acknowledges readiness, receives the committed route, and streams output with request and token sequence identifiers so that the origin can reject stale or duplicated messages.

CHAPTER 4

PROJECT DESCRIPTION

This chapter describes the technology choices and five logical modules. The modules are boundaries within the proposed Android application and native runtime; they are not assumed to be Docker containers or remote services.

4.1 TECHNOLOGIES

The Android client is developed in Kotlin, with coroutine-based tasks for telemetry, discovery, network I/O, and user interaction. A native C++ inference runtime performs model loading, prefill, and decode through a JNI interface. Quantized model assets are stored locally so that a session can operate after external connectivity is removed.

The local communication layer carries capability advertisements, heartbeats, authenticated control messages, chat requests, and streamed tokens. A message schema includes protocol version, peer identity, session identifier, assignment epoch, sequence number, and payload length. Protocol and model versions are validated before a participant becomes eligible to host.

The dashboard uses an Android-native interface. Local bounded storage supports transcripts, calibration constants, and diagnostic records. Exported evaluation logs can be analyzed using offline data-processing tools, but those tools are not part of the serving dependency chain.

The initial implementation avoids distributed training and per-token model partitioning. Its purpose is to assign an already available model to a suitable phone and maintain stable service as device conditions change.

4.2 MODULES

The proposed application is divided into five cooperating modules:

1. Mesh Communication Module;
2. LCI Computation Module;
3. Event-Based Routing Module;
4. Host Inference Module;
5. Dashboard.

Telemetry preprocessing is a shared input stage for LCI computation and event monitoring. Each module exposes explicit success and failure outcomes. A failed probe, a disconnected peer, and a model mismatch are different conditions and must not collapse into an unexplained low score.

4.2.1 MESH COMMUNICATION MODULE

The mesh communication module discovers peers, admits trusted devices, maintains connections, and transports application messages. It provides the routing engine with a registry containing node identity, serving consent, model inventory, capabilities, last-seen time, and connection state.

Discovery and admission. A joining phone advertises its protocol version and supported serving capabilities. The group approves or pairs the peer before accepting requests or private context. Model advertisements include an identifier and content hash. Discovery alone does not authorize a device to receive a conversation.

Heartbeat and freshness. Each participant periodically publishes a compact telemetry snapshot. The router timestamps receipt using its own monotonic clock and checks sequence order. A snapshot older than a configured freshness limit is excluded from new host admission. Missed-heartbeat thresholds and transport failures are logged separately because packet delay is not the same as confirmed device departure.

Framing and streaming. Control messages use bounded frames with explicit type and length. Token streams carry a request identifier, assignment epoch, and token sequence number. The origin only accepts output from its committed host assignment. Partial messages, oversized payloads, and protocol mismatches produce errors rather than undefined state.

Reconnect behavior. A peer reconnects with backoff and re-advertises its capabilities. It must receive the current epoch before resuming service. Reconnecting does not automatically revive an expired assignment or resend a previously completed user request.

Operational boundary. A logical peer mesh can run over a normal local access point. The prototype should not depend on unsupported multi-hop networking or privileged hardware access. Network availability, permissions, and topology are recorded for every evaluation run.

4.2.2 LCI COMPUTATION MODULE

The LCI computation module converts heterogeneous measurements into a comparable suitability estimate. It receives six signals: decode speed, memory headroom, thermal

headroom, queue availability, battery state, and network RTT. All normalized signals have the same orientation: a larger value represents a more favorable condition.

For a beneficial observation x with calibration bounds ℓ and u , the proposed normalization is

$$z(x) = \text{clip}\left(\frac{x - \ell}{u - \ell}, 0, 1\right), \quad u > \ell. \quad (4.1)$$

For cost observations such as RTT and queue delay, the orientation is reversed. Bounds must be declared and versioned rather than silently recomputed from whichever peers happen to be present. Otherwise, the meaning of an 8% score improvement could change merely because a phone joins the group.

Let d_i , m_i , h_i , q_i , b_i , and r_i denote the normalized decode, memory, thermal, queue, battery, and network signals for phone i . A proposed weighted score is

$$\text{LCI}_i = w_d d_i + w_m m_i + w_h h_i + w_q q_i + w_b b_i + w_r r_i, \quad (4.2)$$

where every weight is nonnegative and

$$w_d + w_m + w_h + w_q + w_b + w_r = 1. \quad (4.3)$$

The supplied pipeline uses the weight vector

$$(w_d, w_m, w_h, w_q, w_b, w_r) = (0.30, 0.22, 0.18, 0.12, 0.10, 0.08).$$

Its displayed index is $I_i = 1000 \text{LCI}_i$. Optional unavailable signals can be reweighted; absent safety-critical observations remain subject to conservative admission checks.

Live Compute Index (LCI) telemetry and candidate scoring pipeline

Lightweight probing (under 80 ms) · 1 Hz heartbeat · Dynamic weight normalization

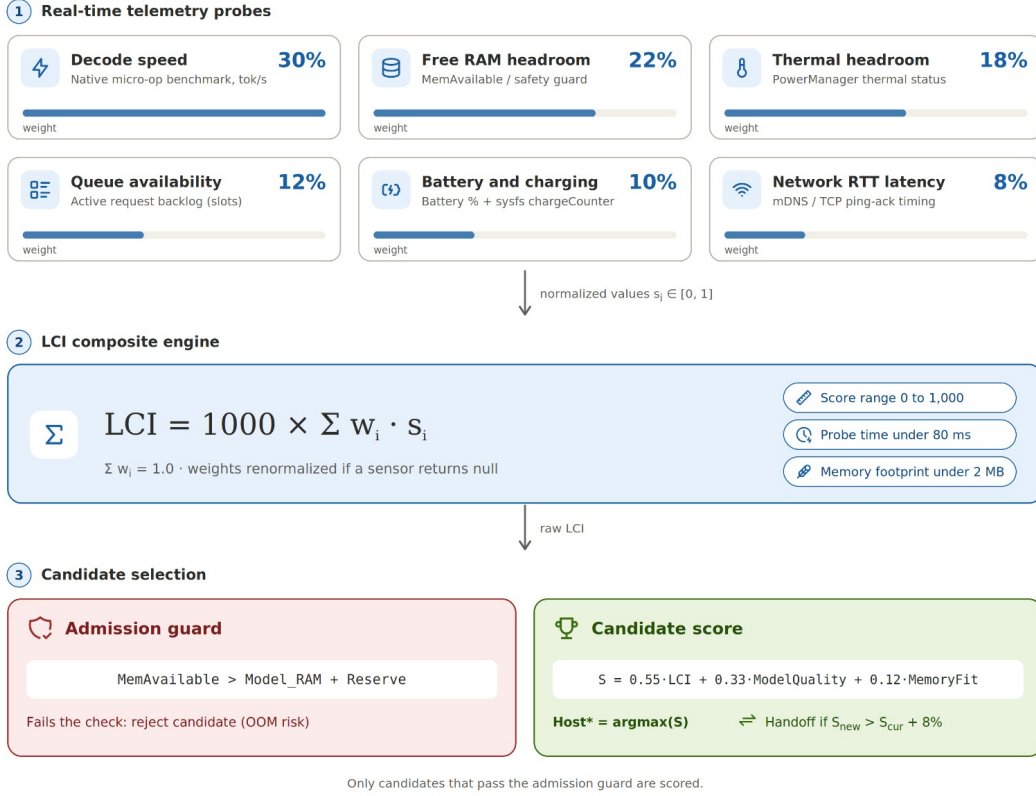


Figure 4.1: LCI Telemetry and Candidate Scoring Pipeline

Figure 4.1 keeps feasibility separate from ranking. A phone is feasible only if it is reachable, trusted, consented, sufficiently fresh, compatible with the requested model, and within configured resource limits. Its free memory must satisfy

$$M_{free,i} \geq M_{weights} + M_{context} + M_{buffers} + M_{reserve}. \quad (4.4)$$

The context estimate depends on model configuration and requested context length. The memory reserve is calibrated conservatively and does not assume that all currently free memory is available to the application.

For a feasible candidate, the supplied pipeline combines the displayed LCI, model quality Q_i , and memory fit F_i on a common $[0, 1000]$ scale. The score is

$$S_i = 0.55I_i + 0.33Q_i + 0.12F_i, \quad i \in \mathcal{F}, \quad (4.5)$$

where $\mathcal{F}$ is the feasible set. Model incompatibility excludes a candidate before scoring. Model quality is constant across compatible hosts in the fixed-model evaluation. The image’s “+8%” shorthand is interpreted as the relative margin formalized below.

Short-term smoothing can suppress sensor noise, but must not hide a critical

thermal or memory event. Stale or unavailable measurements reduce confidence and can trigger conservative exclusion. The dashboard exposes raw observations, normalized terms, weight contributions, and guard outcomes so that an LCI value remains interpretable.

4.2.3 EVENT-BASED ROUTING MODULE

The event-based routing module decides when to revisit an assignment. It consumes the current host state, candidate scores, network events, request outcomes, and user consent changes. It does not automatically migrate a session whenever any score changes.

Elective triggers include a sustained increase in queue delay, loss of thermal headroom below a noncritical threshold, a configured battery participation boundary, or the arrival of a materially better candidate. A persistence window and minimum dwell time suppress transient events.

Forced triggers include confirmed host disconnection, critical safety state, failed memory admission, inference failure, or hosting consent withdrawal. These invalidate the old host and bypass elective stickiness. When no replacement is feasible, the conversation is paused with a recoverable error rather than sent to an unsafe device.

For an eligible current host h and candidate j , the proposed relative stickiness rule is

$$S_j > (1 + \delta)S_h, \quad \delta = 0.08. \quad (4.6)$$

This is an 8% relative improvement, not an addition of 0.08 to a normalized score. If the current score is zero, elective switching also requires a configured absolute improvement floor. Feasibility loss always follows the forced-recovery path.

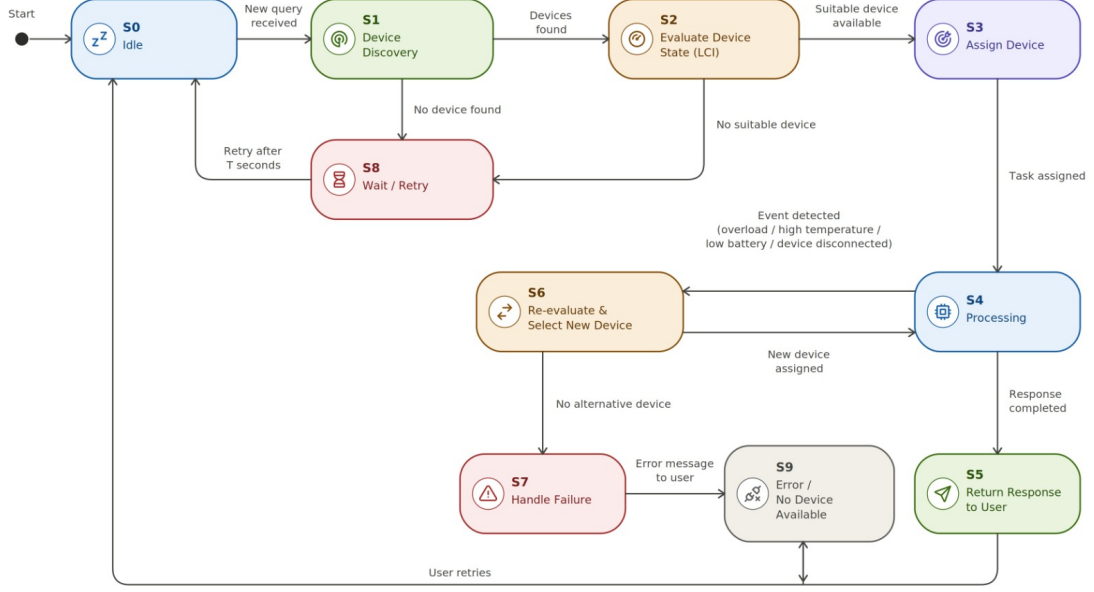


Figure 4.2: Event-Based Routing State Machine

Figure 4.2 follows discovery, scoring, assignment, processing, response delivery, and re-evaluation. Retry and no-device branches distinguish unavailability from completion. The detailed prepare–ready–route protocol commits a route only after readiness confirmation.

The prepare message identifies the session, requested model, context digest, and proposed epoch. The candidate validates compatibility and reserves resources. Ready confirms the admitted state and its expiration time. Route commits the new epoch, after which the origin accepts output only from that assignment. Timers clean up abandoned preparation state.

For elective migration, the preferred transition point is a completed-turn boundary. If the old host fails during generation, the origin preserves completed turns and explicitly treats the partial response as interrupted. It may reissue the unfinished request to the new host, but bit-identical continuation is not assumed. Transferring a native runtime cache between heterogeneous devices is not a baseline requirement.

4.2.4 HOST INFERENCE MODULE

The host inference module runs the local model and exposes a cancellable request interface. Its responsibilities include model validation, memory reservation, context construction, prefill, decode, token streaming, and cleanup. Native resources are released on completion, cancellation, timeout, or failure.

Model readiness. The host checks the exact model hash, tokenizer identity, runtime settings, and permitted context length. A model already resident in memory can reduce preparation delay, but warm state must still pass current memory and safety checks.

Conversation reconstruction. The origin provides the authorized transcript and prompt template. The host tokenizes and reconstructs context locally. This preserves the logical conversation but may introduce prefill delay. The system must distinguish this delay from the routing-control phase when evaluating handoff.

Generation and streaming. Output is sent in ordered chunks with request and epoch identifiers. Backpressure prevents an unresponsive client from causing unlimited buffering. Cancellation reaches the native generation loop, and a cancelled assignment cannot resume output after a newer epoch is committed.

Resource protection. Admission occurs before generation begins and is monitored during execution. Queue capacity and simultaneous context limits are enforced. Critical thermal status or memory pressure triggers cancellation or a refusal to admit new requests according to the configured policy.

Failure reporting. The module distinguishes model-load failure, insufficient memory, invalid context, lost connection, cancellation, and runtime inference error. It reports structured events to the router and user-facing summaries to the dashboard. It does not claim to prevent all device damage or extend battery lifespan without evidence.

The module may support CPU-only inference initially and add compatible accelerator backends later. Hardware acceleration must be tested on the actual devices rather than inferred from advertised SoC capabilities. Native runtime and backend versions are part of the experiment configuration.

4.2.5 DASHBOARD

The dashboard combines the group chat with a transparent view of routing state. It lists connected peers, logical roles, hosting consent, available models, raw telemetry, normalized score contributions, and current eligibility. Unavailable sensor values are visibly distinguished from valid readings.

A conversation panel displays the selected host, assignment epoch, request status, and latest routing reason. Participants can see whether the host was retained because of stickiness, excluded by memory admission, or changed after a disconnect or thermal event. The view must not imply that a high score is a guarantee of future performance.

Controls allow users to enable or disable hosting, choose a battery participation threshold, leave the group, clear retained transcripts, and export sanitized diagnostic

logs. The initial interface should expose conservative defaults rather than require users to understand every weight in the scoring model.

CHAPTER 5

IMPLEMENTATION AND RESULTS

This chapter presents the workflow and evaluation in the supplied diagrams, plots, and tables. Values are transcribed from the uploaded result images, not generated or estimated. The five-phone comparison uses Qwen 0.6B Q8_0, a 2048-token context, four CPU threads, and llama.cpp. The performance source reports 50 runs per configuration with 150 output tokens; the prompt-length source reports 20 repetitions per cell.

5.1 TELEMETRY PREPROCESSING MODULE

Telemetry preprocessing provides validated inputs to the LCI engine and event monitor. A device snapshot includes peer identity, sequence number, model inventory version, raw observations, source information, validity flags, and timestamps. The router records reception time using its own monotonic clock to determine freshness without relying on synchronized phone wall clocks.

5.1.1 TELEMETRY COLLECTION AND PREPROCESSING

The proposed collector reads available memory, supported thermal status, battery charge and charging state, current queue state, observed decode throughput, and network RTT. Observed inference speed is used when valid recent requests exist; a bounded benchmark estimate can be used otherwise, with its age and provenance recorded.

Each observation passes range checks and missing-value handling. Thermal measurements from different devices are not assumed to describe the same physical component. OS thermal status and a validated temperature sensor are separate fields. Missing values do not become artificially favorable scores.

The preprocessing stage aligns observations into snapshots, applies declared normalization bounds, and optionally smooths noncritical variation. Admission checks receive unsmoothed critical signals where needed. Decode measurements distinguish prefill from token generation; network RTT is not substituted for inference service time.

Table 5.1: Device Configuration in the Supplied Five-Phone Evaluation

Phone	Identifier	Processor
Realme GT 6T	RMX3853	Snapdragon 7+ Gen 3
Samsung Galaxy S23 FE	SM-S711B	Exynos 2200
Google Pixel 7 family	Pixel 7 / 7 Pro	Tensor G2
Realme 11 Pro+ 5G	RMX3741	Dimensity 7050
Realme 5 Pro	RMX1971	Snapdragon 712

Source: supplied table, image_5.png. The Pixel entry is ambiguously identified as Pixel 7 / Pixel 7 Pro in the source; it is not counted as two mesh participants.

Table 5.1 identifies the five devices in the supplied evaluation. Device identity and processor are retained without inferring installed memory or OS versions from capability tiers. Controlled comparisons use matching model assets and account for variation in initial conditions.

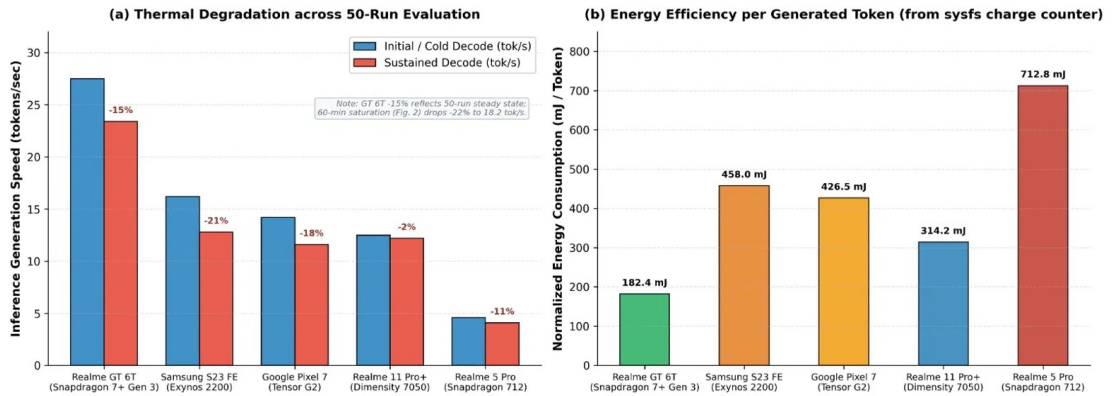
**Figure 5.1:** Thermal Degradation and Source-Reported Energy per Generated Token

Figure 5.1 compares initial and sustained decode speed and charge-counter energy estimates. The source reports 182.4, 458.0, 426.5, 314.2, and 712.8 mJ/token for the GT 6T, S23 FE, Pixel 7, Realme 11 Pro+, and Realme 5 Pro. Charge-to-energy conversion requires voltage and an accounting interval; these estimates are reproduced from the figure, not independently calibrated.

The collector is assessed for overhead, measurement freshness, missing-data frequency, and interference with generation. The proposed sub-80 ms budget applies to the stated lightweight collection/scoring path. It does not include full benchmarking, model loading, or context reconstruction.

5.1.2 IMPLEMENTATION OF LCI COMPUTATION

The LCI implementation applies Equations 4.1–4.5 to a versioned snapshot. The processing order is: validate measurements, evaluate hard admission guards, normalize

supported observations, compute weighted LCI, apply the declared readiness policy, and rank feasible candidates. A stable peer identifier can resolve exact ties without causing random host changes.

The score calculation returns both a scalar and a structured explanation. The explanation includes raw inputs, normalized terms, weights, confidence flags, exclusions, model readiness, and calibration version. This supports debugging and ensures that the dashboard can explain a result without independently recomputing a potentially different score.

Implementation checks should cover normalization bounds, reversed cost metrics, invalid bounds, unavailable sensors, stale snapshots, insufficient memory, incompatible models, and exact ties. A high LCI must never bypass the feasibility set. Weight sensitivity and ablation experiments should compare the full score with variants omitting one signal at a time.

Evaluation metrics. Routing overhead is the elapsed time between a request reaching the routing authority and a valid dispatch decision. TTFT is measured from request submission to the first accepted output token. Decode throughput excludes prefill and is measured as

$$D = \frac{N_{\text{generated}}}{T_{\text{decode}}}. \quad (5.1)$$

The energy-per-token measure is

$$E_{\text{token}} = \frac{E_{\text{measured}}}{N_{\text{generated}}}, \quad (5.2)$$

with the measurement interval and accounting boundary stated. For example, total session energy and incremental inference energy are not equivalent quantities.

Request success is defined before the experiment, including completion, correct assignment, and any deadline requirement. Its reporting rate is

$$R_{\text{success}} = \frac{N_{\text{successful requests}}}{N_{\text{submitted requests}}} \times 100\%. \quad (5.3)$$

Cancelled requests, rejected requests, retry attempts, and partial responses are reported explicitly instead of silently removed from the denominator.

Table 5.2: Reported Inference Throughput and Request Latency

Configuration	Prefill (tok/s)	Decode (tok/s)	TTFT 64 (ms)	TTFT 256 (ms)	150-token latency (s)
Event-based mesh	63.5	22.8	1076	4144	7.62
Static mesh (GT 6T)	58.2	19.8	1174	4695	8.75
Round-robin mesh	24.4	12.4	2755	11120	18.65
GT 6T standalone	64.2	23.4	1045	4230	7.95
S23 FE standalone	36.5	12.8	1838	7378	13.50
Pixel 7 standalone	34.8	11.6	1930	7733	14.80
11 Pro+ standalone	26.8	12.2	2476	9859	14.70
5 Pro standalone	10.4	4.1	6405	26374	42.80

Source: image_5.png; Qwen 0.6B Q8_0, 2048-token context, four CPU threads. TTFT 64 and TTFT 256 denote prompt lengths. Values are retained as reported rather than recomputed from separately aggregated throughput.

Table 5.3: Reported Tail Latency, Thermal Metric, and Request Success

Configuration	p95 Latency (s)	Thermal Metric (%)	Success (%)
Event-based mesh	8.45	3.2	99.4
Static mesh (GT 6T)	11.60	34.0	96.0
Round-robin mesh	44.20	14.8	82.0
GT 6T standalone	9.85	26.5	98.0
S23 FE standalone	28.50	42.0	92.0
Pixel 7 standalone	16.80	18.5	98.0
11 Pro+ standalone	16.10	24.0	97.5
5 Pro standalone	52.40	65.0	60.0

Source: image_5.png. Thermal metric reproduces the source’s “Thermal Throt.” column; its aggregation is not specified. The p95 values measure request turnaround, not routing-control overhead. Fractional success rates are retained without inventing request counts.

Tables 5.2 and 5.3 reconstruct the supplied performance image as readable tables. Static mesh assignment uses a fixed GT 6T host; round robin cycles over five participants. Standalone rows describe individual devices. The adaptive policy’s sustained-service benefit must be distinguished from cold dispatch cost.

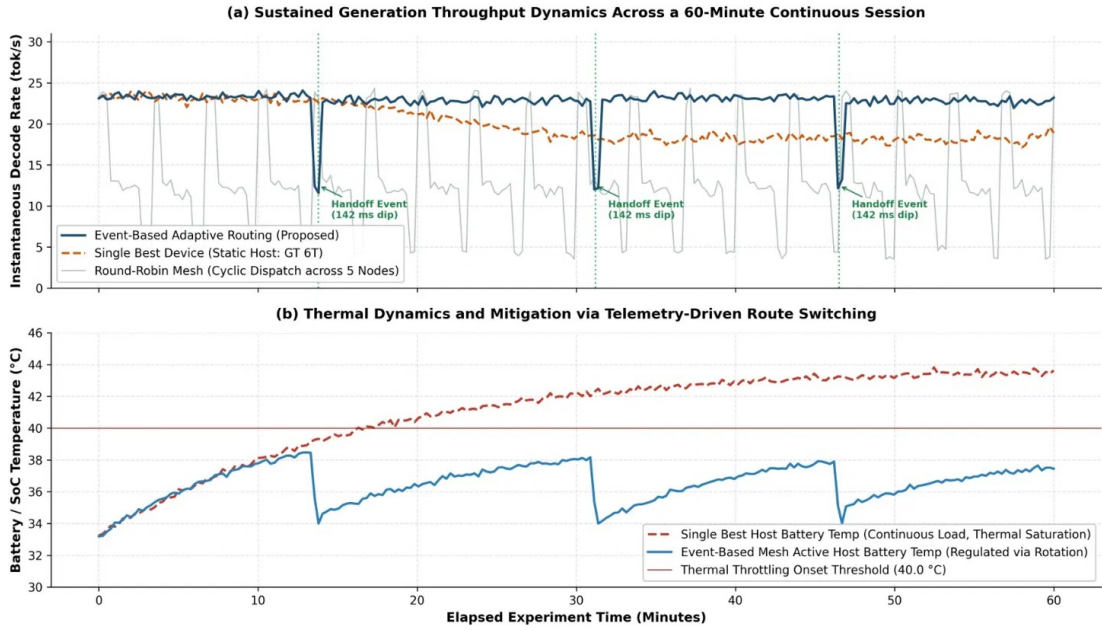


Figure 5.2: Sustained Throughput and Battery-Temperature Dynamics during the Continuous Session

Figure 5.2 shows stable adaptive throughput, declining fixed-host performance, and variable round-robin service. Marked handoffs coincide with reductions in the adaptive host’s battery-temperature trace. Battery temperature is not relabeled as processor temperature; the summaries do not establish behavior outside this workload.

5.1.3 IMPLEMENTATION OF EVENT-BASED ROUTING

The event router maintains explicit session states and commits host assignments with a monotonically increasing epoch. It distinguishes hard failures from elective improvement events and serializes competing handoff attempts so that multiple monitors cannot independently commit different hosts for the same session.

An elective change first checks the current host’s eligibility, the 8% relative margin, the persistence window, and minimum dwell time. It then prepares a feasible candidate without discarding the old assignment. The old host continues to own the session until readiness is validated and a new route is committed. Any reserved resources are released after a failed or expired preparation.

Forced recovery begins when the current assignment is no longer valid. The router chooses a feasible replacement without applying the elective margin. If none exists, the origin retains completed turns, displays the interruption, and waits for a valid retry or participant change. The system must not report a successful handoff merely because it sent a prepare message.

For evaluation, separate handoff timing into

$$T_{\text{handoff}} = T_{\text{control}} + T_{\text{transfer}} + T_{\text{context}} + T_{\text{restart}}. \quad (5.4)$$

The proposed sub-200 ms target is restricted to a declared warm-host routing-control path. The full user-visible interruption includes context preparation and renewed generation, and must be reported separately for cold and warm candidates.

Table 5.4: Reported Time to First Token across Prompt Lengths

Configuration	64 Tokens (ms)	128 Tokens (ms)	256 Tokens (ms)	512 Tokens (ms)
Event-based mesh	1076	2095	4144	8435
Static mesh (GT 6T)	1174	2330	4695	9902
Round-robin mesh	2755	5510	11120	23835
GT 6T standalone	1045	2085	4230	8875
S23 FE standalone	1838	3660	7378	15506
Pixel 7 standalone	1930	3845	7733	16192
11 Pro+ standalone	2476	4920	9859	20986
5 Pro standalone	6405	12925	26374	57780

Source: `image_8.png`; 20 repetitions per cell. TTFT includes prefill, first-token decode, and dispatch overhead; it is not a routing-decision timing.

Table 5.5: Reported Prefill Throughput across Prompt Lengths

Configuration	64 Tokens (tok/s)	256 Tokens (tok/s)	512 Tokens (tok/s)
Event-based mesh	63.5	62.8	61.2
Static mesh (GT 6T)	58.2	55.4	52.1
Round-robin mesh	24.4	23.3	21.6
GT 6T standalone	64.2	61.2	58.0
S23 FE standalone	36.5	35.1	33.2
Pixel 7 standalone	34.8	33.5	31.8
11 Pro+ standalone	26.8	26.2	24.5
5 Pro standalone	10.4	9.8	8.9

Source: `image_8.png`; Qwen 0.6B Q8_0. Round-robin throughput is reported as an effective rate across the rotating nodes.

Tables 5.4 and 5.5 distinguish prompt-dependent TTFT from prefill throughput. At 512 tokens, adaptive TTFT is 8435 ms, versus 9902 for static assignment and 23835

for round robin. The source associates the Realme 5 Pro’s 57780 ms result with memory pressure near its stated 4 GB limit, reinforcing context-dependent memory admission.

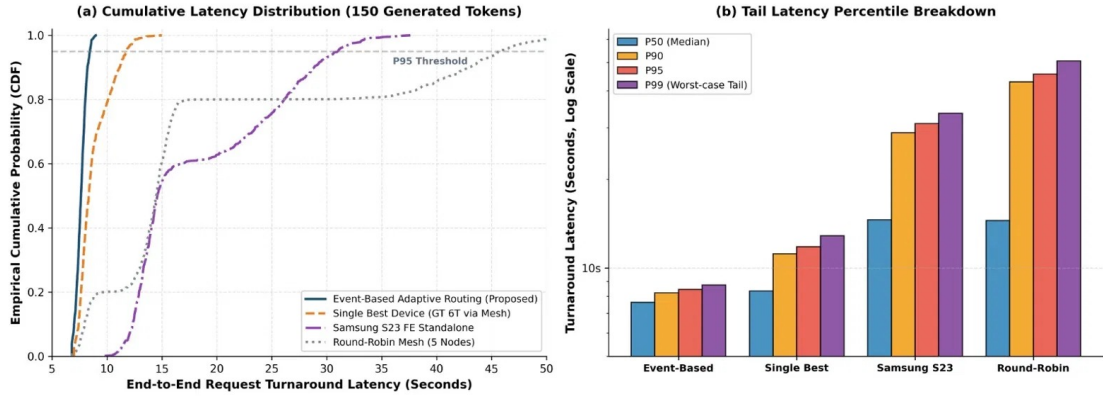


Figure 5.3: Request-Latency Cumulative Distribution and Tail Percentiles

Fault injection should cover host disconnection, stale telemetry, memory admission failure, critical thermal events, readiness timeout, duplicated control messages, delayed old-host tokens, and router replacement. Each test checks the visible outcome, retained transcript, committed epoch, cleanup behavior, and whether the system generates more than one authoritative response.

Synthetic events can validate control logic, but they do not substitute for measured physical thermal behavior or power consumption. A simulated thermal event is labeled as such. Similarly, one short uninterrupted run cannot establish a population-level MTBF or a long-term battery-health benefit.

5.2 DASHBOARD

The implementation dashboard must display the active session, host identity, peer eligibility, latest score contributions, and reason for the most recent routing decision. It should show heartbeat age and sensor availability so that a missing reading is not mistaken for an ideal resource state.

The interface also needs clear states for preparing a candidate, rebuilding context, streaming a response, retaining the current host, recovering from failure, and waiting for an available host. Users can disable hosting or withdraw from the group without requiring access to internal router settings.

The diagnostic view presents host selection, admission outcomes, handoff events, and recovery states without exposing private conversations or pairing credentials. Metrics retain their native units in the tables; normalized summaries are supplementary.

Table 5.6: Reported Endurance, Battery Temperature, and Failure Outcomes

Configuration	Duration (min)	Peak (°C)	Mean (°C)	Reported Outcome
Event-based mesh	60	38.4	35.6	Normal; thermally balanced
Static mesh (GT 6T)	60	43.8	41.2	Throttling; decode decrease of 22%
Round-robin mesh	60	37.8	34.6	Frequent timeouts on low-tier node
S23 FE standalone	28	43.8	41.8	Timeout; CPU reported at 400 MHz
5 Pro standalone	14	38.6	36.8	Low-memory termination; available memory below 180 MB

Source: `image_9.png`. The 28-minute run ends in a timeout and the 14-minute run in a low-memory failure. Temperatures follow the battery-temperature interpretation in `image_6.png`; they are not SoC-temperature measurements.

Table 5.7: Reported Handoff Outcomes and Availability Statistics

Configuration	Handoff (ms)	Handoff Success (%)	Reported MTBF (h)
Event-based mesh	142	100.0	> 24.0
Static mesh (GT 6T)	Not applicable	Not applicable	2.5
Round-robin mesh	Not applicable	Not applicable	0.9
S23 FE standalone	Not applicable	Not applicable	0.45
5 Pro standalone	Not applicable	Not applicable	0.22

Source: `image_9.png`. “Not applicable” denotes a policy without the adaptive handoff metric, not an empty result. The source labels its last column “Availability MTBF” without observation exposure or failure counts; these are source-reported statistics, not independently established MTBF estimates. A single 60-minute run cannot establish a > 24-hour MTBF. The 142-ms handoff value does not separately establish cold-loading or full-context reconstruction time.

**Multi-Dimensional System Performance Trade-off Analysis
(Normalized Metric Space [0-100], Revealing Real Dispatch Costs)**

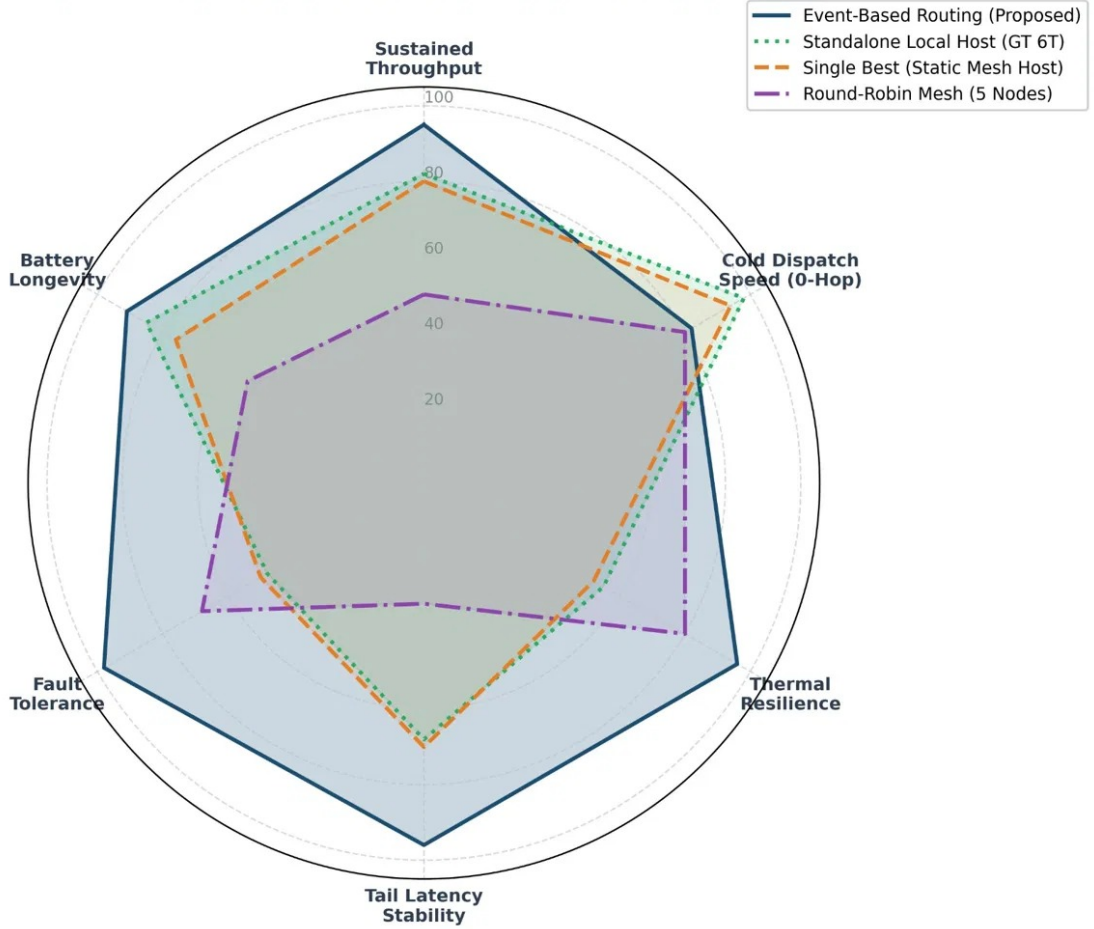


Figure 5.4: Source-Provided Multi-Dimensional Performance Trade-off Analysis

The radar plot is a supplementary summary, not a replacement for raw units and uncertainty. Throughput, latency, and energy measure different quantities; their normalization must not obscure a poor result on a critical safety or reliability axis.

Result discussion. The adaptive mesh reports 22.8 tokens/s sustained decode, compared with 19.8 for static assignment and 12.4 for round robin. Its p95 request latency is 8.45 s versus 11.60 and 44.20 s. Standalone GT 6T remains faster for 64-token TTFT (1045 versus 1076 ms), showing a dispatch-cost trade-off rather than superiority on every metric.

The summaries support a workload-specific comparison. Raw logs, exact request denominators, energy conversion details, and long-exposure reliability records are not reconstructed from images; no unreported measurements are invented.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This report presents a coherent design for device-aware routing of offline group AI chat across a local network of smartphones. It defines the problem of selecting a sustainable host from heterogeneous devices and explains why static assignment, indiscriminate rotation, and single-signal scheduling may be insufficient for long sessions.

The proposed solution combines a transparent LCI with model compatibility checks, memory admission, telemetry freshness, user consent, and thermal and battery participation limits. Its five logical modules support peer communication, score computation, event-based routing, local inference, and user-visible explanations. Serving is separated from idle-time calibration so that profiling does not unnecessarily compete with active requests.

An event-based policy distinguishes elective optimization from forced recovery. The proposed relative 8% stickiness margin reduces switching for marginal score improvements, while a prepare–ready–route protocol controls assignment changes. Completed conversation turns remain under origin control, and interrupted partial output is handled explicitly rather than presented as seamless state migration.

The design removes the unrelated cloud-training and Reddit moderation assumptions from the supplied template. It relies on compatible local models and a trusted local network, while acknowledging that offline operation alone does not guarantee privacy, supported telemetry, or bounded end-to-end handoff time.

The supplied evaluation reports 22.8 tokens/s sustained decode, 8.45 s p95 request latency, and 99.4% request success for adaptive routing. The endurance summary reports a 38.4 °C peak battery temperature and 142 ms handoff latency. These results favor adaptive selection for sustained service in the reported workload; the summaries alone do not establish long-term reliability or battery-health benefits.

6.2 FUTURE WORK

Future development should prioritize implementation evidence and controlled evaluation before adding more complex scheduling features. The main directions are:

1. **Reproducibility:** Archive runtime revisions, device inventories, sanitized request

logs, event counts, and measurement scripts for independent reproduction of the results.

2. **Weight and threshold calibration:** Tune normalization bounds, LCI weights, freshness limits, persistence windows, and dwell times using held-out workloads rather than the same runs used to claim improvement.
3. **Context-transfer optimization:** Evaluate compatible context caching, incremental transcript synchronization, and warm standby hosts while accounting for memory and battery overhead.
4. **Multiple simultaneous sessions:** Add per-context memory reservations, fair queueing, user participation budgets, and scheduling under concurrent requests.
5. **Router resilience:** Validate authority replacement, epoch reconciliation, reconnect behavior, and safe operation during network partitions.
6. **Broader device support:** Test additional Android devices, supported accelerator backends, different model sizes, and platform-specific telemetry limitations.
7. **Stronger trust mechanisms:** Explore detection of inconsistent peer reports and stronger integrity guarantees while recognizing the limits of software-only telemetry.
8. **Long-term energy and health studies:** Evaluate longer workloads and documented power measurements before making claims about battery aging or device longevity.

These extensions should preserve the project’s central priorities: transparent decisions, safe admission, user consent, and reliable offline service. Improvements are accepted on the basis of measured outcomes, not assumed benefits from additional complexity.

REFERENCES

- [1] T. Tan and G. Cao, “Thermal-Aware Scheduling for Deep Learning on Mobile Devices with NPU,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 10706–10719, 2024. DOI: 10.1109/TMC.2024.3379501.
- [2] X. Gao, “TAS: A Temperature-Aware Scheduling for Heterogeneous Computing,” *IEEE Access*, 2023. DOI: 10.1109/ACCESS.2023.3281839.
- [3] J. Zhang, Y. Wang, T. Huang, F. Dong, W. Zhao, and D. Shen, “Thermal-Aware on-Device Inference Using Single-Layer Parallelization with Heterogeneous Processors,” *Tsinghua Science and Technology*, vol. 28, no. 1, pp. 82–92, 2023. DOI: 10.26599/TST.2021.9010075.
- [4] Y. Gao, Z. Zhang, P. K. Donta, C. K. Dehury, X. Wang, D. Niyato, and Q. Zhang, “Optimizing Multi-DNN Inference on Mobile Devices through Heterogeneous Processor Co-Execution,” arXiv:2503.21109, 2025.
- [5] P. Tummalapalli, S. Arayakandy, R. Pal, and K. Kundan, “LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load,” arXiv:2603.23640, version 2, 2026.
- [6] Z. Chen, B. Zhu, J. Wang, H. Shin, A. Nallanathan, and D. Niyato, “Network Edge Inference for Large Language Models: Principles, Techniques, and Opportunities,” arXiv:2604.22906, 2026.
- [7] J. Zhao, Y. Song, S. Liu, I. G. Harris, and S. Abdu Jyothi, “LinguaLinked: Distributed Large Language Model Inference on Mobile Devices,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, Volume 3: System Demonstrations*, pp. 160–171, 2024. DOI: 10.18653/v1/2024.acl-demos.16.
- [8] D. Lim, W. Lee, W.-T. Kim, and I. Joe, “DRL-OS: A Deep Reinforcement Learning-Based Offloading Scheduler in Mobile Edge Computing,” *Sensors*, vol. 22, no. 23, article 9212, 2022. DOI: 10.3390/s22239212.
- [9] Y. Tang, H. Huang, W. Zhan, G. Min, Z. Duan, and Y. Lei, “Battery-Care Resource Allocation and Task Offloading in Multi-Agent Post-Disaster MEC Environment,” arXiv:2312.15380, 2023.
- [10] P. Lin, “Fuzzy based priority aware task scheduling optimization for mobile edge computing environments,” *Scientific Reports*, vol. 15, article 39998, 2025. DOI: 10.1038/s41598-025-23690-9.
- [11] Mesh-LLM contributors, *mesh-llm: Distributed AI/LLM for the people*, project repository and documentation, accessed October 1, 2026. Repository identifier: Mesh-LLM/mesh-llm on GitHub.